Lezione 4 - Il Setup del Programmatore

Dal notebook al terminale

Matteo Caldana

Mani in pasta: creiamo il nostro pacchetto

Ambiente di sviluppo online:
Una macchina virtuale di Google https://shell.cloud.google.com/

Ambiente di sviluppo locale:
Il cloud e la maggior parte di server funzionano con Linux, quindi ci allineamo:

- su MacOS tutto ok (Linux e MacOS sono fratellastri)
- su Windows, usiamo una virtual machine: seguite le istruzioni di installazione di WSL e lanciate il comando `wsl --install Ubuntu-24.04` da Terminale.

Editor (*cioè dove scriviamo il codice*): installate VS Code

Bibliografia:
Python Packages, by T. Timbers & T. Beuzen, https://py-pkgs.org/welcome

Perché abbandoniamo i Notebook?

I Jupyter Notebook sono fantastici per l'esplorazione dei dati, ma hanno dei limiti:

- **Esecuzione non lineare**: è facile confondersi con l'ordine delle celle.
- **Variabili globali**: è difficile tenere traccia di tutte le variabili globali .
- **Difficoltà di automazione**: non sono pensati per girare su un server h24.
- **Collaborazione**: è difficile tenere traccia delle modifiche in files `.ipynb` .

Oggi impariamo a scrivere script `.py` e a gestire l'ambiente di lavoro.

Linguaggio di Programmazione

Un linguaggio non è il notebbok, ne il software che installate, ne il codiche che scrivete ma un **insieme di regole** sintattiche e semantiche.

Due approcci principali:

1. **Linguaggi Compilati** (es. C++, Rust, Go):
    - Il codice viene trasformato interamente in un file binario (eseguibile) prima dell'esecuzione.
    - Performance altissime, ma ciclo di sviluppo più lento.
2. **Linguaggi Interpretati** (es. Python, JavaScript):
    - Il codice viene letto ed eseguito riga per riga da un altro programma.
    - Facilità d'uso e flessibilità.

L'Interprete Python

Python è un linguaggio interpretato. Per farlo girare, usiamo l'**interprete**.

- L'interprete non è il linguaggio, è un **eseguibile** (un programma) che legge il vostro testo e lo traduce in azioni per la CPU. L'interprete deve essere implementato seguendo le regole definite dal linguaggio.
- Quando scrivete `python script.py` nel terminale, state dicendo all'interprete: *"Leggi questo file e fai quello che c'è scritto secondo le regole di Python"*.

L'importanza del terminale (CLI)

Il terminale non è "roba da hacker", è lo strumento di lavoro principale.

- **Interfaccia Grafica (UI)**: limitata a ciò che lo sviluppatore ha previsto.
- **Terminale (CLI)**: accesso diretto a ogni comando del sistema operativo.

Ogni azione che fate con il mouse (creare una cartella, spostare un file) è in realtà un comando che viene inviato al sistema. Imparare i comandi significa poter **automatizzare** il lavoro.

Navigazione Base

Un comando è un eseguibile situato in una posizione specifica del vostro disco (usate il comando `which` per vedere dove si trova e.g., `which pwd`).

- `pwd` (Print Working Directory): "Dove mi trovo?". Mostra il percorso della cartella corrente.
- `ls` (List): "Cosa c'è qui?". Elenca file e cartelle.
  - Esempio con opzioni: `ls -la` (mostra anche i file nascosti e i dettagli).
- `cd` (Change Directory): "Spostami lì".
  - Esempio: `cd documenti/progetti`

Gestione File e Cartelle

- `mkdir` (Make Directory): Crea una cartella. `mkdir nuovo_progetto`
- `touch`: Crea un file vuoto. `touch main.py`
- `mv` (Move): Sposta o rinomina. `mv vecchio.py nuovo.py`
- `cp` (Copy): Copia file o cartelle. `cp file.txt copia_file.txt`
- `rm` (Remove): Elimina. *Attenzione: non c'è il cestino!* `rm file.txt`
- `cat`: Visualizza il contenuto di un file nel terminale. `cat main.py`

Risorsa extra per approfondire: Lezione PACS sul terminale

`git`: La Macchina del Tempo

`git` serve a tenere traccia di ogni modifica al codice e collaborare con altri. Pensatelo come un **cloud** (GDrive, OneDrive, ...) fatto per programmatori e usato con il terminale. Servirebbero ore per fare un corso su `git`, vi spiego solo la punta dell'iceberg: gli essenziali per lavorare **da soli**.

Il ciclo di lavoro fondamentale:

1. `git init`: Inizializza una nuova "macchina del tempo" nella cartella corrente.
2. `git add .`: "Prepara" i file modificati (li mette nello stage).
3. `git commit -m "messaggio"`: Salva ufficialmente lo stato dei file con un commento.
4. `git push`: Invia i tuoi salvataggi su un server remoto (es. GitHub).

`git`: Collegare GitHub

Per salvare il codice "nel cloud", dobbiamo seguire i seguenti step.

Una tantum: dobbiamo creare una password sicura sul nostro PC che GitHub puo' chiedere senza che noi facciamo nulla. (1) create una chiave ssh (creazione della password) ed (2) aggiungetela al profilo (dire a GitHub come chiedercela).

Per ogni nuovo progetto, creiamo uno spazio apposito:

1. Create un nuovo repository su GitHub.com. `https://github.com/new`
2. Nel terminale (i comandi esatti ve li da GitHub appena dopo la creazione):

```
git remote add origin https://github.com/user/repo.git
git branch -M main
git push -u origin main
```

`git`: Controllare la storia

Come facciamo a sapere cosa è successo nel passato?

- `git status`: Il comando più usato. Vi dice quali file sono stati modificati e cosa è già pronto per il commit.
- `git log`: Mostra la lista cronologica di tutti i commit fatti.
  - Ogni commit ha un **Hash** (un codice univoco come `a1b2c3d`).
  - Mostra chi ha fatto la modifica, quando e con quale messaggio.

Consiglio: usate messaggi di commit chiari, il "voi del futuro" vi ringrazierà.

`git`: Tornare indietro nel tempo

Il vantaggio di Git è poter rimediare agli errori.

- **Annullare modifiche non ancora salvate**:
  - `git restore nome_file.py` (Riporta il file all'ultimo commit).
- **Viaggiare nel passato**:
  - `git checkout <hash_commit>`: Vi permette di guardare il codice come appariva in quel momento.
- **Reset pesante**:
  - `git reset --hard <hash_commit>`: Cancella tutto quello fatto dopo quel commit e torna lì. *Usare con estrema cautela!*

`git`: Non vogliamo file grossi!

Git è fatto per il **codice** (testo), non per i dati pesanti, video o cartelle di sistema.

Il file `.gitignore`

È un file di testo che elenca cosa Git deve ignorare.

- Evita di caricare cartelle giganti come `.venv/` o `__pycache__/`.
- Evita di caricare file segreti (es. `.env` con le vostre password).

Esempio di contenuto `.gitignore`:

```
__pycache__
.venv/
data/*.csv
```

Python Environments

Come i linguaggio umano, anche il Python **evolve** e circa ogni anno esce una nuova versione. Anche i moduli possono cambiare (es. risoluzione di un bug, implementazione di una nuova funzione...).

Per progetti complessi, puo' essere necessario utilizzare **versioni esatte** di Python o di un certo modulo.

Perché non installare tutto "globalmente" sul PC? **Dependency hell**: Quando importiamo un modulo non c'e' la versione. Se progetti diversi usano moduli che necessitano versioni diverse, uno si romperà.

Dobbiamo creare degli **ambienti isolati** (scatole chiuse) per ogni progetto.

Strumenti Moderni: uv e micromamba

micromamba

- Versione leggerissima e veloce di Conda, scritta in C++.

uv

- Scritto in Rust, è un approccio nuovo ed incredibilmente veloce.

uv: L'astro nascente

- **Sito Ufficiale**: https://docs.astral.sh/uv/
- **Installazione**: `curl -LsSf https://astral.sh/uv/install.sh | sh`

Puoi creare un ambiente isolato specificando la versione di Python desiderata (la scaricherà lui se manca):

- `uv venv --python 3.12` (Crea un venv con Python 3.12)
- `uv pip install numpy` (Installa numpy)
- `uv run main.py` (Esegue lo script nel contesto del progetto)

Micromamba: Potenza e Leggerezza

- **Sito Ufficiale**: https://mamba.readthedocs.io/
- **Installazione**: `"${SHELL}" <(curl -L micro.mamba.pm/install.sh)`

Essentials:

- `micromamba create -n mio_env python=3.11`
- `micromamba activate mio_env`
- `pip install numpy`
- `python main.py`

I fondamentali - Moduli

Creaiamo la directory del nostro pacchetto `pyaim` e mettiamoci due files

```
pyaim
├── aimmodule.py
└── main.py
```

Con contenuto

```python
# aimmodule.py
def hello_world():
    print("Hello World!")
# main.py
import aimmodule
if __name__ == "__main__":
    a = 1
    print(dir())
    print(dir(aimmodule))
    print(type(aimmodule))
    aimmodule.hello_world()
```

I fondamentali - Moduli

Un **namespace** è una mappatura da nomi a oggetti. Prima, per esempio, abbiamo aggiunto il nome `a` (un numero intero). Con `dir()` possiamo ispezionare un namespace. Senza argomenti, `dir()` si refiresce al namespace corrente.

Il punto centrale dei namespace è che quando un modulo viene importato utilizzando l'istruzione `import`, viene creato un **oggetto modulo** che ha a sua volta un namespace. Con `type()` possiamo controllare che `aimmodule` è difatti un oggetto `module`.

`__name__` è una variabile creata da Python nel namespace che si chiama `__main__` solo nel modulo lanciato direttamete (indipendentemente dal nome del file). Serve quindi per eseguire codice solo se si lancia un file direttamente, non se importato come modulo.

I fondamentali - Pacchetto

I pacchetti sono semplicemente un insieme di uno o più moduli. In genere sono strutturati come una directory (il pacchetto) contenente uno o più file .py (i moduli) e/o sottodirectory (che chiamiamo sottopacchetti). Un file speciale denominato `__init__.py` viene utilizzato per indicare a Python che una directory è un pacchetto (anziché una semplice directory sul computer). Parleremo più approfonditamente di `__init__.py`

```
pyaim
├── main.py
├── aimmodule.py
└── subaimpkg
    ├── __init__.py
    └── subaimmodule.py
```

I fondamentali - Pacchetto

Quando si importa un pacchetto o un modulo, Python lo cerca nell'elenco predefinito di directory definito in `sys.path`.

```
>>> import sys
>>> sys.path
```

Questa lista è solitamente composta da:

- La directory da dove state eseguendo lo script
- `PYTHONPATH`: una variabli ambientale di sistema
- La directory dove è installato Python.
- La directory dove sono installati i pacchetti con `pip` (detti `site-packages`)

I fondamentali - Pacchetto

La differenza tra un **modulo** e un **pacchetto** è che i pacchetti vengono importati come oggetti modulo che hanno un attributo `__path__`.

Python utilizza l'attributo `__path__` del pacchetto per cercare quell'elemento, anziché i percorsi in `sys.path`.

Ciò significa che quando si digita `import subaimpkg.subaimmodule`, Python cerca prima un modulo o un pacchetto chiamato `subaimpkg` nell'elenco dei percorsi di ricerca definiti da `sys.path`. Se è un pacchetto, cerca quindi un modulo o un sottopacchetto plotting utilizzando `subaimpkg.__path__` come percorso di ricerca (anziché `sys.path`).

La struttura del pacchetto professionale

```
pkg
├── src                          # <-- non necessario ma consigliato
│   └── pkg
│       ├── __init__.py
│       ├── module1.py
│       └── subpkg
│           ├── __init__.py
│           └── module2.py
├── README.md
└── pyproject.toml
```

Indicazioni per i nome del pacchetto e moduli: **corti**, **tutto minuscolo**, **no underscore** e **significativi**.

Intra-package references

Spesso server usare in un modulo il codice di un altro modulo

```
src
└── package
    ├── __init__.py
    ├── moduleA.py
    ├── moduleB.py
    └── subpackage
        ├── __init__.py
        └── moduleC.py
```

Un import "intra-package" può avvenire traimite un import **assoluto** o **relativo**.

Import	Assoluto	Relativo
Import from `moduleA` in `moduleB`	`import package.moduleA`	`import .moduleA`
Import from `moduleA` in `moduleC`	`import package.moduleA`	`import ..moduleA`
Import from `moduleC` in `moduleA`	`import package.subpackage.moduleC`	`import .subpackage.moduleC`

Opinioni forti: Ha senso usare solo import assoluti.

Installiamo il nostro pacchetto

Per ora abbiamo potuto usare le funzioni di `aimmodule.py` e `subaimmodule.py` perchè si trovano nella stessa cartella di `main.py`. Tuttavia, se vogliamo passarlo a qualcuno o usarlo più liberamente conviente installarlo.

Installare un pacchetto signifaca mettere il contenuto di `{package}` in una cartella che sta in `sys.path`, per convenzione `site-packages/`, insieme a una cartella `{package}-{version}.dist-info` che contiene le informazioni di installazione.

`pip` è lo strumento che ci aiuta ad automatizzare questo proceso. A `pip` basta uno di questi files per completare l'installazione:

- `requirements.txt`, una lista di dipendenze che ci servono (es. `pandas`)
- `setup.py`, script con istruzioni, sconsigliato può essere **malevolo**
- `pyproject.toml`, l'abbiamo visto prima nella struttura del pacchetto, **consigliato**.

pyproject.toml

```
[build-system]
# These are the tools needed to actually turn your code into a package.
requires = ["setuptools", "wheel"]
# This tells pip which 'engine' to use to run the build.
build_backend = "setuptools.build_meta"
[project]
# The name of your package (how it will appear on PyPI).
name = "pyaim"
# Use Semantic Versioning (Major.Minor.Patch).
version = "0.1.0"
# A short tagline for your project.
description = "A small project that does big things."
# Who to blame (or thank) for the code.
authors = [{name = "Your Name", email = "you@example.com"}]
# The minimum Python version required to run your code.
requires-python = ">=3.8"
# External libraries your code depends on, with version requirement.
dependencies = ["pandas"]
[project.optional-dependencies]
# Extra packages installed via `pip install ".[dev]"`
dev = ["pytest", "black"]
```

Usiamo pip (Package Installer Python)

- **Installare un pacchetto:** `pip install <nome>` cercato su internet (PyPI)
- **Rimuovere un pacchetto:** `pip uninstall <nome>`
- **Vedere cosa hai installato:** `pip freeze` usabile come **requirements.txt**

Quando sviluppiamo un pacchetto e vogliamo testarlo mentre programmiamo `pip install -e .`

- `-e` editabile: crea un **link** tra `site_packages/` e la posizione del pacchetto
- `.` specifica la posizione del pacchetto (cartella corrente)

Risultato: Ogni modifica che fai al codice sorgente è immediatamente attiva senza dover reinstallare il pacchetto ogni volta.

Il file init

Il file `__init__.py` può essere, e **spesso è**, lasciato vuoto e utilizzato solo allo scopo di identificare una directory come pacchetto. Quando un pacchetto viene importato, il file `__init__.py` viene eseguito, quindi può anche essere utilizzato per aggiungere oggetti allo spazio dei nomi del pacchetto, fornire documentazione e/o eseguire altro codice di inizializzazione.

Due semplici esempi

```
# read version from installed package
from importlib.metadata import version
__version__ = version(__name__)

# populate package namespace
from pyaim.subaimpkg.subaimmodule import hello_world as hello_world_v2
```

Unit test

I test assicurano che il codice funzioni come previsto. Usiamo **pytest**.

- Crea una cartella `tests/` nella root.
- I file devono iniziare con `test_`.
- Le funzioni devono iniziare con `test_`.

Esempio di test (`tests/test_core.py`):

```
from pyaim.aimmodule import my_sum
def test_somma():
    assert my_sum(1, 1) == 2
```

Esegui i test nella home del pacchetto: `pytest`

Scrivere codice per "umani": Type Hinting

In Python le variabili sono dinamiche, ma questo è un rischio nei grandi progetti. I **Type Hints** sono "contratti" che firmi con il tuo codice.

```
def calcola_area(raggio: float) -> float:
    return 3.14 * (raggio ** 2)
```

- `raggio: float` dice cosa ti aspetti in ingresso.
- `-> float` dice cosa restituirai.

Attenzione: Come spesso accade in Python, puoi rompere questi contratti quando vuoi senza alcuna ripercussione.

Pro: esistono programmi (chiamati "linter") che sfruttano questi indizi per risolvere errori (es. Ruff o Mypy) o dare suggerimenti.

Formatting: L'estetica è funzione

Non perdere tempo a discutere se usare apici singoli o doppi, o quanti spazi mettere. Usa un **Formatter automatico**.

Black: "The Uncompromising Code Formatter"

Black formatta il codice in modo univoco. Non hai scelta (quasi), ed è un bene.

- **Installazione**: modulo `pip install black`
- **Comando**: `black .`
- **Filosofia**: "Il codice del team deve sembrare scritto dalla stessa persona".
- **Pro**: Niente discussioni su spazi e tabulazioni. Il codice diventa piu' leggibile.

What's next?

- CI/CD
- Documentazione e pagina web
- Releases e versioning
- Licenze sul codice
- Collaborazione e pull-requests

E molto altro!